Faculty of Engineering - University of Ruhuna

Software Requirement Specification

ELABS - Smart Laboratory Management & LMS Platform

Version 1.0

6/20/26

VERSION AND APPROVALS

VERSION HISTORY

Version #	Date	Revised By	Reason for change
		1.0	20-Jun-2026
ELABS Dev Team	Initial release for academic evaluation and submission.		

This document has been approved as the official Software Requirements Document for ELABS, and accurately reflects the current understanding of business requirements. Following approval of this document, requirement changes will be governed by the project's change management process, including impact analysis, appropriate reviews and approvals.

DOCUMENT APPROVALS

Approver Name	Project Role	Signature/Electronic Approval	Date
		Dr. Lecturer	Academic Evaluator / Lecturer
	20-Jun-2026	Prof. Head	EIE Department Head
	20-Jun-2026		

PROJECT DETAILS

Project Name	ELABS - Smart Laboratory Management & LMS Platform
Project Type	Academic Capstone Initiative
Project Start Date	01-Mar-2026
Project End Date	20-Jun-2026
Project Sponsor	Department of Electrical and Information Engineering, Faculty of Engineering, University of Ruhuna
Primary Driver	Safety Compliance & Workflow Efficiency
Secondary Driver	AI Student Assistant & Automated Checkouts
Division	Faculty of Engineering
Project Manager/Dept	Undergraduate EIE Capstone Group

OVERVIEW

This document defines the high level requirements ELABS. It will be used as the basis for the following activities:

- Creating solution designs
- Developing test plans, test scripts, and test cases
- Determining project completion
- Assessing project success

DOCUMENT RESOURCES

Name	Business Unit	Role
Frontend Developer	packages/web & packages/mobile	Web & Mobile Client UI Engine (Next.js, Expo)
Backend Developer	packages/api & packages/shared	API Gateway, Relational Database & Socket Server
AI Specialist	packages/ai	FastAPI Chatbot Server & RAG Engine
Vision Engineer	packages/vision	OpenCV Pipelines, YOLOv8 & ByteTrack Node
EIE Department Staff	Department Office	System Users, Administrators & Technicians

GLOSSARY OF TERMS

Term/Acronym	Definition
ELABS	Smart Laboratory Management & LMS Platform
LMS	Learning Management System
RAG	Retrieval-Augmented Generation (context-aware AI response)
YOLOv8	You Only Look Once version 8 (real-time object detection model)
ByteTrack	Multi-object tracking algorithm used for persistent ID tracking
RTSP	Real-Time Streaming Protocol (used for CCTV stream captures)
RBAC	Role-Based Access Control
JWT	JSON Web Token (used for user authentication)
Socket.IO	Real-time bidirectional event-based communication library
Ollama	Local open-source LLM runtime interface

Contents

INTRODUCTION

1.1 Purpose

This document specifies the software requirements for ELABS (Smart Laboratory Management & LMS Platform) version 1.0. ELABS is a comprehensive, full-stack monorepo system designed to modernize laboratory access, manage inventory check-outs, and provide real-time safety surveillance for the Department of Electrical and Information Engineering (EIE) at the Faculty of Engineering, University of Ruhuna.

1.2 Document Conventions

This document follows the standard IEEE 830-1998 format for Software Requirements Specifications. Typographical conventions include standard Helvetica body text (10pt), bold headings for structure (Heading 1 at 18pt, Heading 2 at 13pt), courier monospace for code symbols and package directories, and italics for instruction placeholders.

1.3 Intended Audience and Reading Suggestions

This document is written for project evaluators, academic supervisors, software engineers, QA testers, and laboratory staff (technicians and lecturers). For evaluators, we suggest starting with the Overview and the Database Schema, followed by Section 3 (Functionality and Use Cases) and Section 6 (Architecture). For developers and testers, Section 3 and Section 5 (Interfaces) are the most critical.

1.4 Product Scope

ELABS is a modern, full-stack monorepo platform designed for advanced academic laboratory surveillance, inventory check-outs, and learning management (LMS) at the Department of Electrical and Information Engineering, Faculty of Engineering, University of Ruhuna. It integrates real-time computer vision (YOLOv8 tracking, pose estimation, and hazard detection), a database-connected AI Assistant (RAG and local LLM), a Next.js web application, and an Expo React Native mobile client.

1.5 References

The references used for ELABS include:

1. Next.js Developer Documentation (https://nextjs.org)
2. Expo React Native SDK Documentation (https://docs.expo.dev)
3. Ultralytics YOLOv8 Documentation (https://docs.ultralytics.com)
4. OpenCV-Python Tutorials (https://docs.opencv.org)
5. Ollama Model Library & API (https://ollama.com)
6. Socket.IO API Documentation (https://socket.io)

OVERALL DESCRIPTION

The ELABS system requirements are shaped by the physical layout of the EIE laboratories, the need for automated student attendance tracking, safety compliance guidelines, and inventory audits. The platform must scale to support high-throughput scanning during lab hours, provide low-latency video streaming, and ensure strict role-based data isolation.

2.1 Product Perspective

ELABS is a new, self-contained monorepo project structured with npm workspaces to unify multiple client and backend services. It comprises web frontends (Next.js), mobile frontends (Expo), Express gateways, FastAPI AI service endpoints, and FastAPI Vision streaming nodes. This modular architecture allows the heavy computer vision and LLM components to run on separate server threads while maintaining synchronous states in a shared MySQL database.

2.1.1 System Interfaces

ELABS interfaces with: (1) Local IP CCTV Cameras via RTSP stream captures; (2) Local Ollama Model Services hosting Llama-3/Mistral models via HTTP requests on port 11434; and (3) The MySQL Database Engine utilizing connection pooling and migrations.

2.1.2 Interfaces

Specify:

(1) The logical user interfaces of the system are designed with high usability and clarity in mind. The web application features a clean, responsive admin dashboard built in Next.js, featuring a 3-column vision command center, interactive LMS checklists, and chat chips. The mobile client features a tab-based navigation flow optimized for one-handed operation, featuring camera scanner overlay indicators and swipeable notification cards.

(2) All the aspects of optimizing the interface with the person who must use the system

This is a description of how the system will interact with its users. Is there a GUI, a command line or some other type of interface? Are there special interface requirements? If you are designing for the general student population for instance, what is the impact of ADA (American with Disabilities Act) on your interface?

2.1.3 Hardware Interfaces

Hardware interfaces include: (1) Network IP/CCTV Cameras: Fetching H.264 video streams over RTSP channels; (2) Mobile Camera Modules: Interfacing via Expo Camera to execute QR code and barcode scans; (3) Host GPU/VRAM: Utilized by YOLOv8 worker threads and Ollama local LLM for model acceleration.

2.1.4 Software Interfaces

The system integrates with the following customer-specified software interfaces:

1. MySQL Database Server (Mnemonic: INT-DB-MYSQL, Spec: SPEC-INT-001, Version: 8.0, Source: Oracle):

• **Purpose:** Relational persistent store for user RBAC, student check-ins, inventory tracking, checklist completions, and timetables.

• **Format:** Standard SQL queries (DDL, DML, DQL) transmitted over TCP port 3306 using the MySQL Client/Server Protocol.

2. Socket.IO Engine (Mnemonic: INT-WS-SOCKETIO, Spec: SPEC-INT-002, Version: 4.7, Source: Socket.IO Project):

• **Purpose:** Real-time, low-latency event channels to broadcast safety alerts, sync YOLO occupancy counts, and push QR scanning confirmations.

• **Format:** JSON payloads serialized over WebSocket connections (ws:// or wss://) on port 4000.

3. Docker Engine (Mnemonic: INT-DEP-DOCKER, Spec: SPEC-INT-003, Version: 24.0+, Source: Docker Inc.):

• **Purpose:** System containerization platform to isolate microservices (AI, vision, API gateways) and standardize on-premise deployments.

• **Format:** CLI runtime configuration and docker-compose configurations managed via daemon API socket commands.

2.1.5 Communications Interfaces

Communication interfaces utilize: (1) HTTPS/HTTP REST protocols for stateless CRUD operations, (2) WebSockets (ws://) for streaming YOLO annotation coordinates to frontends, (3) Socket.IO (WSS/WS) namespaces for private message portals and real-time alarms, and (4) TCP/IP for internal microservice API calls.

2.1.6 Memory Constraints

The system operates under the following memory constraints: (1) The Vision Service requires a minimum of 4GB RAM (CPU-only YOLOv8 tracking) or 2GB VRAM (GPU-accelerated); (2) The AI Service requires a minimum of 16GB system RAM or 8GB dedicated VRAM to hold Llama-3 8B model parameters in memory without performance degradation; (3) The database service requires 500MB of storage for up to 10,000 active student and inventory records.

2.1.7 Operations

Normal operations include daily student check-ins, automated inventory tracking, live camera occupancy syncs, and AI chat queries. Special operations include: (1) System-wide safety alert broadcasts when fire/smoke is detected, (2) Overdue borrow alarms scheduled to run at

08:00 AM daily, and (3) Database snapshots and log rotations scheduled during low-occupancy hours (12:00 AM).

Modes and Periods of Operations:

• **Academic & Research Modes:** The system supports active laboratory sessions, open-access research slots, and administrative inventory audits.

• **Interactive Operations:** Active human-system interactions occur during EIE laboratory operating hours (08:00 AM to 05:00 PM, Monday-Friday), including student checklist check-ins, inventory transactions, and AI assistant prompt queries.

• **Unattended Operations:** Automated background processes operate 24/7, including continuous CCTV frame processing, YOLOv8 pose classification alerts, database maintenance scripts, and the daily 08:00 AM overdue notifications.

• **Data Processing Support:** Frame-by-frame camera scanning and localized LLM inference tasks are executed concurrently on dedicated server threads.

• **Backup and Recovery:** Automated daily database snapshots and log rotations are executed at 12:00 AM (midnight) to minimize CPU load during low-occupancy periods, with snapshots stored in local secondary storage.

2.1.8 Site Adaptation Requirements

Site adaptation requires calibrating the YOLOv8 camera boundaries for the 8 major EIE laboratories: Electric Machines, High Voltage, Electronics, Electronics Workshop, Communication, Computer and Information, Networking, and Undergraduate Project Dev labs. This involves mapping physical boundary polygons in the vision config files to define the active monitoring zones.

Hardware and Environmental Adaptations:

• **IP Camera Installation:** Overhead security cameras supporting RTSP streaming must be mounted in each laboratory to provide clear visual coverage of boundary zones.

• **GPU Server Station:** An on-premise workstation equipped with a dedicated NVIDIA GPU (minimum 8GB VRAM) must be set up inside the EIE department server room to run local AI and vision models.

Software Adaptations:

• **Schema Setup:** The ELABS database schema must be initialized on the department's MySQL 8.0 server and populated with initial student registry rosters and equipment catalogs before deployment.

• **Vision Calibration:** Calibration boundary coordinates (polygons mapping the physical lab entrance/exit zones) must be configured in the vision service's JSON configuration files to ensure accurate attendance logging.

2.2 Product Functions

The major functions performed by the ELABS platform include:

1. Automated Attendance & Laboratory Entry Tracking: Uses CCTV cameras and YOLOv8 object detection to track student presence and automatically log attendance records when they cross physical lab boundaries.

2. Real-time Occupancy & Safety Surveillance: Identifies student activities and safety violations (e.g. running, overcrowding, or fire/smoke hazards) using deep learning pose tracking and HSV fallback filters.

3. Smart Inventory Cataloging & Borrowing (QR-based): Allows laboratory technicians to manage equipment catalogs and execute instant check-outs and check-ins using QR code and barcode scanning on mobile client interfaces.

4. Offline AI Assistant & RAG Guidance: Answers student queries regarding laboratory procedures, course schedules, and item locations using an offline Intent Router and a local Llama-3/Mistral LLM linked to RAG databases.

5. Administrative Scheduling & Checklists: Enables lecturers to publish semester timetables, assign weekly checklists to lab courses, and audit student grades and compliance logs.

2.3 User Classes and Characteristics

The user classes identified for the ELABS platform include:

• **Students:** Interact with laboratory timetables, borrow inventory equipment, complete weekly checklists, and submit queries to the AI chatbot assistant.

• **Technicians:** Manage equipment catalogs, approve checkout transactions, scan returned items, and log instrument wear conditions.

• **Lecturers:** Organize course modules, configure schedules, assign checklists, and audit attendance logs.

• **Administrators:** Configure system parameters, register users, and monitor overall safety alarm logs.

2.4 Operating Environment

Client frontends run on modern web browsers (Chrome, Firefox, Safari) and mobile operating systems (Android 10+, iOS 15+). Backend services run on Node.js v18+ and Python 3.10+ on Windows Server 2022 or Ubuntu 22.04 LTS. The shared database runs on MySQL 8.0.

2.5 Design and Implementation Constraints

Constraints include: (1) Local LLM and YOLOv8 models must run offline on local hardware due to university internet reliability and privacy restrictions; (2) Mobile scanning must function under poor network conditions within laboratory basements; (3) The database schemas must strictly adhere to university relational database patterns.

Key Engineering Constraints:

• **Hardware & Computing:** Deep learning inference runs locally on the department server workstation. Model throughput requires at least 4GB of GPU VRAM for the vision service and 8GB of VRAM for the AI local LLM.

• **Network Isolation:** The system must operate fully offline on the university intranet due to network reliability limitations in laboratory basement areas. Barcode and QR code scanning must cache data locally and synchronize on reconnection.

• **Safety & Reliability:** Fallback HSV color-space filters must activate autonomously if deep-learning occupancy or fire classification threads encounter frame-drop or runtime failures.

• **Security & Hashing:** Database security must comply with university storage standards, using Bcrypt password hashing.

• **Tracking Constraints:** Real-time video processing must maintain a frame rate of at least 10fps to ensure tracking accuracy.

2.6 Stakeholders

	Stakeholders
1.	Department Head & Faculty Administrators
2.	Laboratory Instructors & Lecturers
3.	Laboratory Technicians
4.	Students (Undergraduates & Researchers)

The following comprises the internal and external stakeholders whose requirements are represented by this document:

1. Department Head & Faculty Administrators: Oversee EIE laboratory utilization, safety compliance, and overall student attendance statistics.

2. Laboratory Instructors & Lecturers: Define academic checklists, coordinate schedules, and verify student course accomplishments.

3. Laboratory Technicians: Manage equipment inventory catalogs, authorize borrowing checkouts/check-ins, and inspect tool conditions.

4. Students (Undergraduates & Researchers): Access timetables, log entry attendance, complete lab checklists, request inventory loans, and utilize the offline AI chatbot.

2.7 User Documentation

The following user documentation components will be delivered along with the ELABS software platform:

• **ELABS Administrator Configuration Guide:** Complete setup manuals detailing server configuration, Docker daemon configurations, and database initialization.

• **Laboratory Technician Mobile Guide:** Handheld scanner operations, QR code checking flows, offline caching syncs, and hardware calibrations.

• **Student Interactive Help Console:** Interactive onboard guiding prompts and prompt suggestion chips embedded directly within the Next.js web application.

2.8 Assumptions and Dependencies

#	**Assumptions**
	Students have access to active student accounts and university-issued QR/Barcodes.
	Laboratory facilities are equipped with network IP cameras supporting RTSP streaming.
	Local server hardware has GPU resources sufficient to host Llama-3/Mistral models.
	Laboratory schedules and class lists are synchronized and updated in the main database.
	Students carry their mobile devices to use the scanner or checkout features.
	Technicians have authorized login credentials for mobile inventory operations.
#	**Constraints**
	The AI Assistant must run fully offline to meet student privacy and data sovereignty regulations.
	Emergency fallback for fire and smoke must operate via OpenCV HSV color-space filters without internet access.
	Mobile client check-outs must support offline sync in basement lab environments.
	Real-time video processing must maintain a frame rate of at least 10fps to ensure tracking accuracy.
	Database security must comply with university storage standards, using Bcrypt password hashing.

	All inventory items must be uniquely tagged with a physical barcode matching the database record.

Assumed factors and dependencies that affect the requirements of the ELABS platform include:

• **Student Registry:** Students carry their mobile devices to use the scanner or checkout features, and have active accounts and university-issued QR/Barcodes.

• **RTSP Camera Feeds:** Laboratory facilities are equipped with network IP cameras supporting reliable H.264 video streams over RTSP.

• **GPU Accelerators:** Local server hardware is equipped with dedicated NVIDIA GPU resources sufficient to host Llama-3/Mistral models and YOLOv8 pipelines.

• **Roster Synchronization:** Laboratory timetables and student enrollment class lists are pre-synchronized in the database before the start of each semester.

• **Mobile Networking:** Mobile client check-outs have access to local LAN connectivity for regular syncs or are pre-authorized with offline transaction caches.

• **Credentials:** Technicians have authorized login credentials for mobile inventory operations.

2.9 Apportioning of Requirements.

The requirements for the ELABS platform are apportioned across development cycles as follows:

• **Delayed / Future Deliverables:** Automated physical turnstile access-control gate integrations, direct chemical hazard sensor hardware interfaces, and multi-campus inter-university data synchronization portals.

• **Core Phase 1 Release:** Real-time CCTV safety tracking, MySQL database schemas, Express API gateway, next.js student/technician dashboards, and basic QR checkout scanning capabilities.

FUNCTIONALITY REQUIREMENTS

This section details the functional requirements of ELABS, categorized by the three main application layers: Web Application workspace, Mobile client scanning workflows, and the Vision/AI backend services.

2.10 3.1 Real-Time CCTV Surveillance & Activity Classification

2.10.1 Description and Priority

This feature processes RTSP camera feeds to count student occupancy and classify activities (standing, walking, running) to detect safety hazards. Priority: High.

2.10.2 Stimulus/Response Sequences

1. System starts and establishes RTSP capture queues.
2. OpenCV captures frames and passes them to YOLOv8 threads.
3. YOLOv8 detects students and ByteTrack persists tracking IDs.
4. Pose estimation classifies running behavior.
5. Event counts are sent to Express `/sync-occupancy` to update database state.
6. Annotated frames and coordinates are pushed to the Web client over WebSockets.

2.10.3 Functional Requirements

REQ-1.1: The system shall capture RTSP streams from the 8 EIE laboratories at a minimum rate of 12fps.
REQ-1.2: The system shall classify student poses using COCO 17-keypoint models.
REQ-1.3: The system shall trigger visual running warning alerts if student displacement speed exceeds 2.5m/s.
REQ-1.4: The system shall activate HSV fire/smoke backup heuristics if deep learning models fail.
Each requirement in ELABS is uniquely identified by a prefix indicating the component (REQ-WEB, REQ-MOB, REQ-AI, REQ-VIS) followed by a hierarchical numeric tag (e.g., REQ-WEB-1.1). This tracking enables clean traceability from requirements to integration tests.

2.10.4 User Stories

User story ID: US-01
User Role: Student
Description: As a student, I want my presence in the laboratory to be automatically detected by the vision system so that my attendance is logged without manual check-in.
Acceptance Criteria: Given that the student is registered, when they enter the lab boundaries and their face/ID is tracked by the camera, then their attendance record is updated in the database and a green checkmark appears on the web console.
Supporting Information/Reference: EIE Laboratory attendance protocol.
Output: Automated attendance log in database (attendance_records table).
Dependencies: Camera feed active, OpenCV parser online.
Integration: Syncs with Express API /attendance/sync-occupancy.

2.10.5 Use Case Narratives

Use Case ID:	UC-01		
Use Case Name:	Borrow Laboratory Equipment via QR Scan		
Created By:	Frontend & Backend Developers	Last Updated By:	ELABS Dev Team
Date Created:	10-Jun-2026	Date Last Updated:	20-Jun-2026

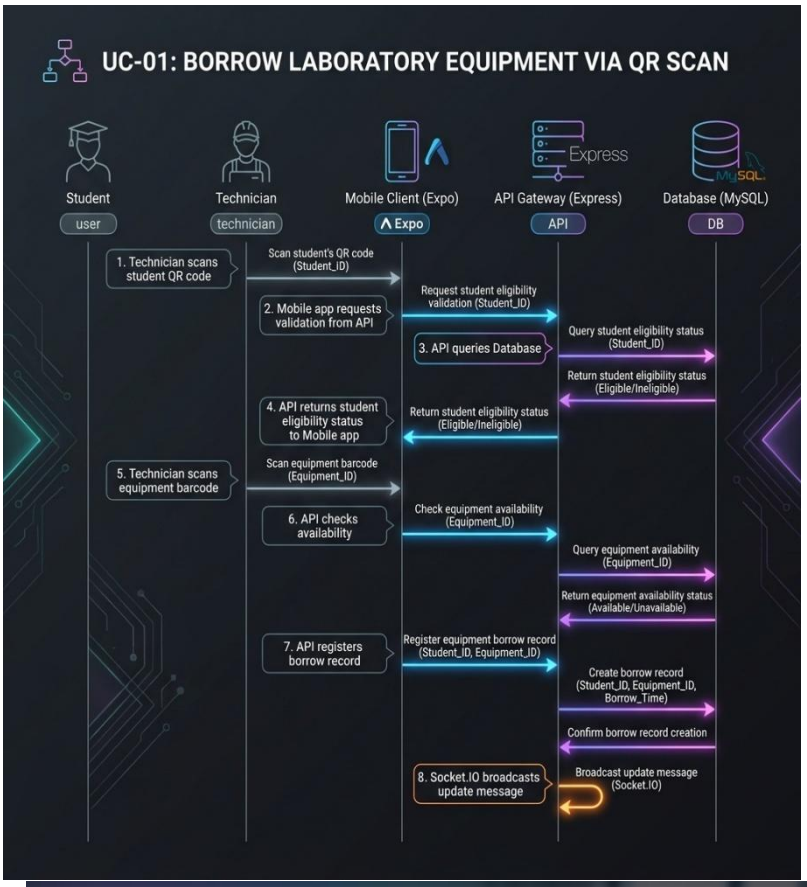
Actors:	Student, Laboratory Technician
Description:	A student requests to borrow an instrument. The technician scans the equipment QR code using the handheld mobile client, verifies the student's eligibility, and confirms the checkout, updating the database status and sending a confirmation socket event.
Preconditions:	1. The student is registered in the database. 2. The equipment item has a valid barcode/QR code and is marked as "AVAILABLE" in the inventory. 3. The technician is logged into the mobile client.
Postconditions:	1. The equipment item status is updated to "BORROWED". 2. A new borrow record is created in the database with the student's profile ID and technician's user ID. 3. The active borrows list on the student's dashboard is updated in real time.
Normal Course:	1. Student requests to borrow an item and presents their digital identity QR code. 2. Technician opens the Expo Mobile client and scans the

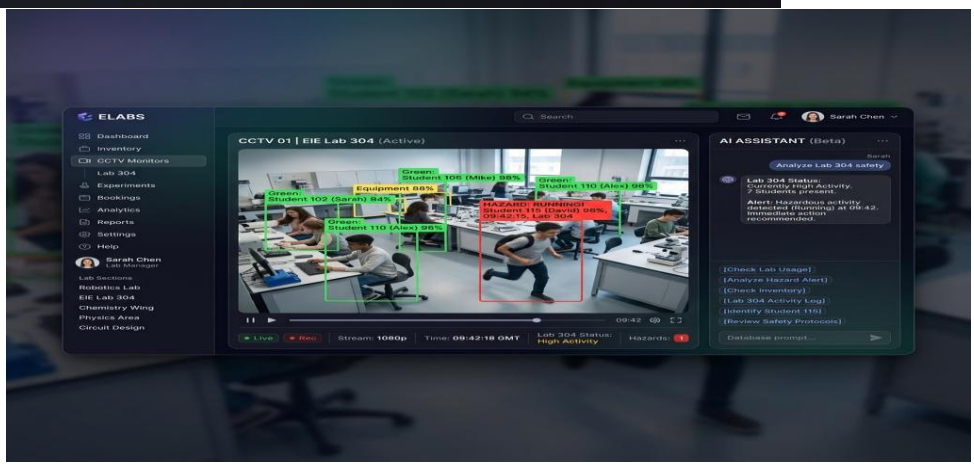
	student's ID. 3. Mobile app displays the student's active status and profile. 4. Technician selects "Scan Borrow" and scans the barcode/QR code of the instrument. 5. System validates that the item is available and has no active holds. 6. System validates that the student has no overdue borrows. 7. Technician clicks "Confirm Borrow". 8. System updates the status in the MySQL database, issues a success Socket.IO message, and displays a confirmation toast on both mobile and web clients.
Alternative Courses:	UC-01.AC.1: Student has active overdue borrows. In Step 6, if the system detects that the student has an outstanding overdue record, a warning is displayed and the borrow action is blocked until the overdue item is returned.
Exceptions:	UC-01.EX.1: Scan failure or invalid QR code. In Step 4, if the scanned QR code cannot be decoded or does not exist in the database, the mobile app displays an "Item Not Found" error and prompts the technician to retry or enter the equipment ID manually.
Includes:	None
Priority:	High
Frequency of Use:	Dozens of times per day during laboratory slots
Business Rules	A student can borrow at most 3 items simultaneously. Checked-out items must be returned within 14 days.
Special Requirements:	The camera scanning must operate in low lighting conditions (using device torch toggle).
Assumptions:	Mobile device has active network access to the API server.
Notes and Issues:	Ensure inventory tags are printed clearly.

The following sections contain the official Use Case Narratives for the primary system operations of the ELABS platform, illustrating actor interactions, system preconditions, postconditions, and alternative execution flows.

2.10.6 Sequence Diagram

2.10.7System Mock-up Screens





2.11 3.2 AI Assistant & RAG Context Engine

The ELABS platform includes an advanced offline AI Assistant that allows users to interact with EIE laboratory data using natural language. The FastAPI backend employs a Regex

Intent Router to parse structured queries (e.g. inventory location, occupancy counts) and execute parameterized SQL statements directly on the database with 100% accuracy. General questions and experiment procedures are resolved using a Retrieval-Augmented Generation (RAG) system linked to local PDF manuals and processed by local LLMs (Llama-3/Mistral).

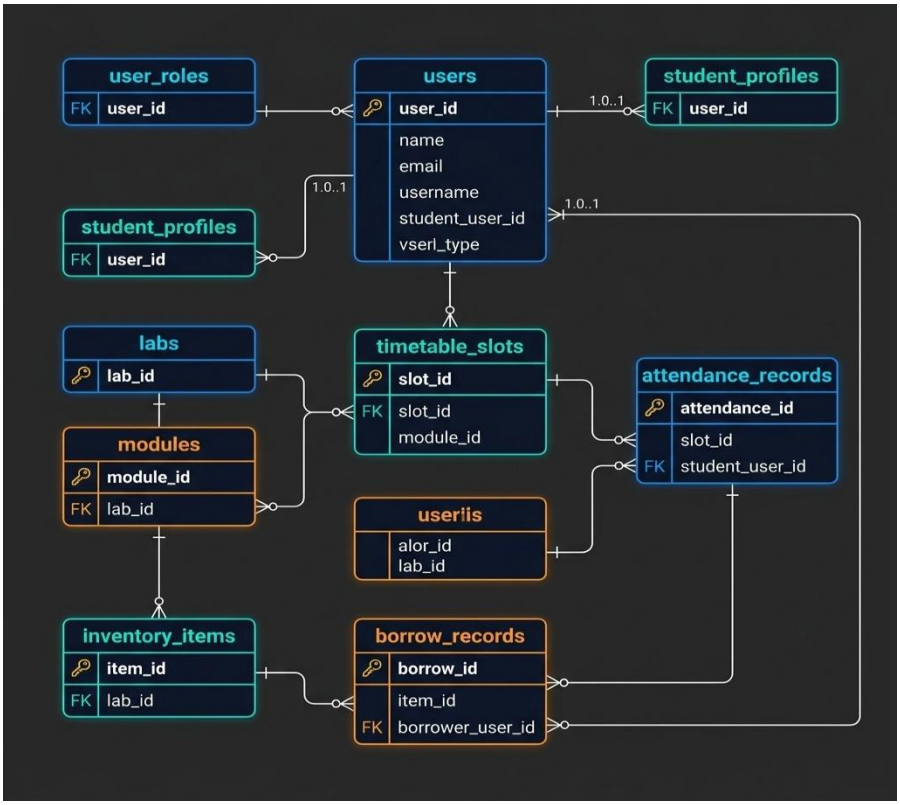
2.12 Logical Database Requirement

The ELABS database schema handles entities for users, roles, timetables, and borrowings. Relationships represent student borrows and check-in logs. Referential integrity rules ensure cascades on deletion.

- Types of information used by various functions
- Frequency of use
- Accessing capabilities
- Data entities and their relationships
- Integrity constraints
- Data retention requirements

If the customer provided you with data models, those can be presented here. ER diagrams (or static class diagrams) can be useful here to show complex data relationships.

Sample


user_roles
FK user_id
users
user_id
name
email
username
student_user_id
vserl_type
1.0..1
student_profiles
FK user_id
1.0..1
student_profiles
FK user_id
1.0..1
labs
lab_id
timetable_slots
slot_id
FK slot_id
module_id
attendance_records
attendance_id
slot_id
FK student_user_id
modules
module_id
FK lab_id
userlis
alor_id
lab_id
inventory_items
item_id
FK lab_id
borrow_records
borrow_id
item_id
FK borrower_user_id

NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

Performance bounds require: (1) Frame-to-canvas rendering latency of YOLO bounding boxes must remain under 150ms over local networks; (2) AI Assistant response generation latency must be under 3.5 seconds for RAG and intent SQL execution; (3) Express API must handle up to 50 requests per second with response times under 50ms.

3.2 Safety Requirements

Safety requirements dictate: (1) Real-time hazard monitoring for fire and smoke utilizing HSV fallback algorithms; (2) Immediate Socket.IO broadcast of hazard events to all active client dashboards; (3) Enforcement of occupancy limits in labs to comply with EIE building safety guidelines.

3.3 Security Requirements

Security constraints include: (1) Secure authentication using JWT and Bcrypt hashing with forced reset on first onboarding; (2) Role-Based Access Control (RBAC) ensuring only authorized technicians can perform inventory transactions; (3) Anonymized vision coordinates, ensuring no student facial features are recorded or stored.

3.4 Software Quality Attributes

The system prioritizes Usability (clear interface cues and chat assistant prompt chips), Reliability (100% uptime for local intent query routers), and Interoperability (seamless integration between next.js, Expo, Express, and FastAPI microservices).

ID	Characteristic	H/M/L	1	2	3	4	5	6	7	8	9	10	11	12
1	Correctness	H												
2	Efficiency	M												
3	Flexibility	M												
4	Integrity/Security	H												

5	Interoperability	H												
6	Maintainability	M												
7	Portability	L												
8	Reliability	H												
9	Reusability	M												
10	Testability	H												
11	Usability	H												
12	Availability	H												

Definitions of the quality characteristics not defined in the paragraphs above follow.

- Correctness - extent to which program satisfies specifications, fulfills user's mission objectives
- Efficiency - amount of computing resources and code required to perform function
- Flexibility - effort needed to modify operational program
- Interoperability - effort needed to couple one system with another
- Reliability - extent to which program performs with required precision
- Reusability - extent to which it can be reused in another application
- Testability - effort needed to test to ensure performs as intended
- Usability - effort required to learn, operate, prepare input, and interpret output

3.5 Business Rules

Business rules mandate: (1) Students can only check out equipment if they have no active overdue borrows; (2) Technicians must verify the item condition before scanning the checkout completion; (3) Lecturers can only access attendance logs for modules they are assigned to.

OTHER REQUIREMENTS

Database schemas must adhere to relational normalization standards up to 3NF, utilizing foreign key integrity constraints for cascading deletions of attendance records and session checklists.

4.1 Online documentation features are integrated into the web client, displaying dynamic API schemas and descriptive tooltips next to inventory and computer vision controllers.

The ELABS system includes built-in onboarding guides, hoverable help tips on complex dashboards, and an automated API reference console. For developers and administrators, a complete online setup guide is maintained inside the project repository.

4.2 Purchased Components

All libraries and dependencies used in ELABS are open-source and do not require purchasing commercial licenses. Key open-source elements include Next.js (MIT License), Expo (MIT License), Express (MIT License), OpenCV (Apache 2 License), YOLOv8 (AGPL-3.0 License / Commercial for proprietary use), and Ollama (MIT License). There are no third-party restrictions.

4.3 Interfaces

The interfaces of the ELABS platform follow clean REST API design conventions and real-time Socket.IO event namespaces. The web client runs on port 3000, Express API gateway on port 4000 (providing authentication, inventory database logic, and WS broadcasters), FastAPI AI Assistant service on port 8001, and FastAPI Vision stream analyzer on port 8002.

4.3.1 User Interfaces

The ELABS user interfaces are designed for modern responsiveness, following a strict dark-mode/glassmorphic aesthetic. The Next.js web application is built on top of a unified design system in index.css, featuring three-column layouts, unified notification sidebar modals, and

persistent navigation controls. The mobile frontend utilizes Expo's React Native components optimized for high contrast during mobile camera scanning operations.

4.3.2 Hardware Interfaces

Hardware interfaces for ELABS include: (1) Local IP Cameras: Accessing RTSP video channels over high-speed Ethernet for continuous frame capture; (2) Mobile Camera Module: Utilized by the Expo React Native app to run the camera interface and capture barcodes/QR codes; (3) Dedicated Workstation GPU: Leveraged by YOLOv8 and Ollama model threads to run hardware-accelerated deep learning inference.

4.3.3 Software Interfaces

ELABS integrates multiple software components: (1) MySQL Database (v8.0) used for core transaction storage, accessed via Prisma ORM in the Node gateway and SQLAlchemy in Python; (2) Express Gateway (v4.18) acting as the central router for stateless REST requests and real-time Socket.IO events; (3) FastAPI (v0.100) microservices hosting deep-learning pipelines for vision tracking and AI assistants. Data is shared via TCP sockets, HTTP REST interfaces, and WebSocket data streams.

4.3.4 Communications Interfaces

All network communications in ELABS rely on standard TCP/IP. Client connections utilize HTTP/HTTPS for standard CRUD APIs, and WebSockets (WS/WSS) for continuous low-latency data streams (e.g. YOLO coordinates and real-time safety notifications). Internal microservice communication uses local HTTP channels on custom ports (e.g., FastAPI AI on 8001, Vision on 8002, Express on 4000). Sockets are monitored for disconnects with automatic reconnection routines.

4.4 Licensing Requirements

ELABS does not implement artificial commercial license checks. It requires standard institutional Google or university login credentials to authenticate users. Student usage is restricted during non-academic hours unless pre-approved by a laboratory coordinator or technician.

4.5 Legal, Copyright and Other Notices

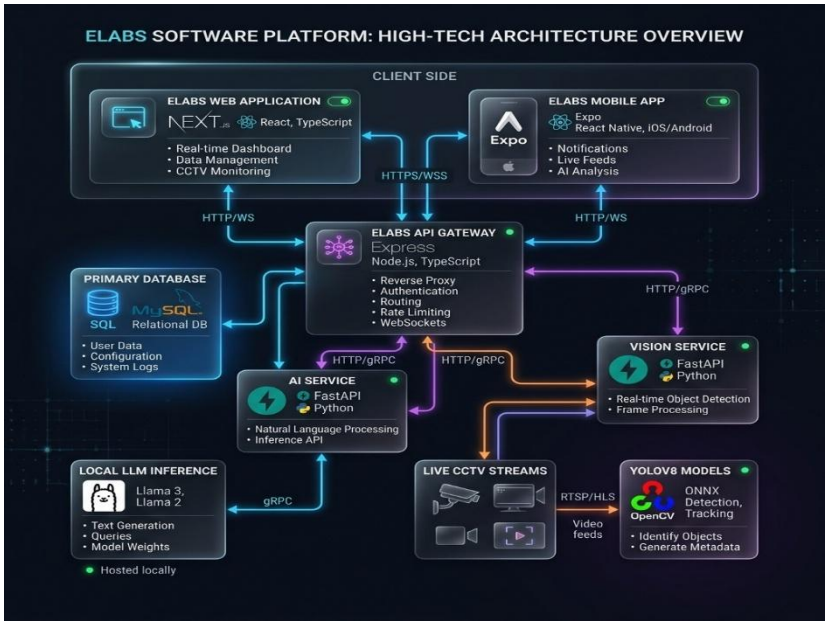
ELABS is developed as an academic capstone project for the Department of Electrical and Information Engineering at the University of Ruhuna. The software is provided 'as is' without warranty of any kind. All intellectual property, trademarks, and logos are property of the University of Ruhuna.

4.6 Applicable Standards

The ELABS system complies with the following industry standards: (1) W3C Web Content Accessibility Guidelines (WCAG 2.1) for frontend layout usability; (2) IEEE 830-1998 standards for software requirements specifications; (3) RFC 6455 WebSockets protocol for real-time streaming; and (4) local department safety protocols for laboratory physical access and fire risk warnings.

SYSTEM ARCHITECTURE

The ELABS system architecture is designed as a TypeScript and Python monorepo, separating client user interfaces from core business API gateways and deep learning microservices to ensure modularity and scalability.



5.1 Functional Architecture

ELABS functions are divided into three functional blocks: (1) Authentication & User Management, (2) Laboratory LMS & Timetabling, (3) Real-Time Survelliance (YOLOv8 Counting, Pose tracking) & AI Chat Assistant (RAG context).

5.2 Logical Architecture

The logical architecture follows a client-server microservice model. The web and mobile clients communicate with the Express API gateway, which serves as the entry point and directly manages the MySQL database. Long-running model inferences are delegated to independent Python FastAPI nodes.

5.3 Component Architecture

The components are divided into npm workspaces inside the monorepo: packages/web (Next.js UI components), packages/mobile (Expo Native wrappers), packages/api (Express controllers and middleware), packages/ai (FastAPI + Ollama), packages/vision (FastAPI + YOLOv8 + OpenCV), and packages/shared (DB prisma schema definitions).

5.4 Physical Architecture

Physically, the system is hosted on an on-premise EIE Department workstation equipped with an NVIDIA GPU. IP cameras in the labs stream feeds to this workstation over the local LAN. Students and staff access the server endpoints via web browsers or mobile devices connected to the Faculty intranet.

APPENDIX A: GLOSSARY

Refer to Section 1.5 and the Glossary of Terms table for comprehensive definitions of all technical acronyms used throughout this document.

APPENDIX B: ANALYSIS MODELS

Detailed database schema definitions and system architecture flows are illustrated in the previous sections. The database entity relationships are mapped in detail under the Logical Database Requirement section.

APPENDIX C: TO BE DETERMINED LIST

All requirements have been successfully defined and mapped. There are no remaining TBD (To Be Determined) items in this release of the ELABS specification.